Linux Process Monitoring & Control System

```
$ ./process_monitor
PID   PPID  STATE
1248  1032  R
1372  1248  S
1410  1248  R

signal → SIGTERM
```

Operating Systems and Systems Programming

25CS2104E • 2026–27, Term-I • Section 13 • Team 7

Adithya Vishnubatla • Abhinav Sunkara • T. Harsha Vardhan

Project Overview

A lightweight Linux utility for visibility and controlled process management.

MONITOR

View PID, PPID, process state, CPU usage, memory usage and command details.

CONTROL

Select a process and perform permitted signal-based control operations.

DEMONSTRATE

Apply OS concepts through direct interaction with Linux/POSIX mechanisms.

Problem Statement

The challenge is not a lack of process data—it is making that data clear and actionable.

- Linux systems run a large number of processes simultaneously.
- Process information is distributed across multiple /proc pseudo-files.
- Users need a clear view of resource usage and parent-child relationships.
- Existing utilities such as top or ps provide information, but their internal working can remain a black box for students.
- The project builds a transparent, lightweight utility directly with Linux/POSIX APIs.

DATA IS SCATTERED

/proc/[pid]/stat • /proc/[pid]/status • /proc/[pid]/cmdline

NEED

Readable process information + controlled action

RESULT

A focused process monitoring and control utility

Project Objectives

01

Monitor

Display essential process information: PID, PPID, state, CPU and memory usage.

02

Track

Observe basic CPU/memory usage and process states.

03

Control

Perform controlled process-management operations using Linux signals.

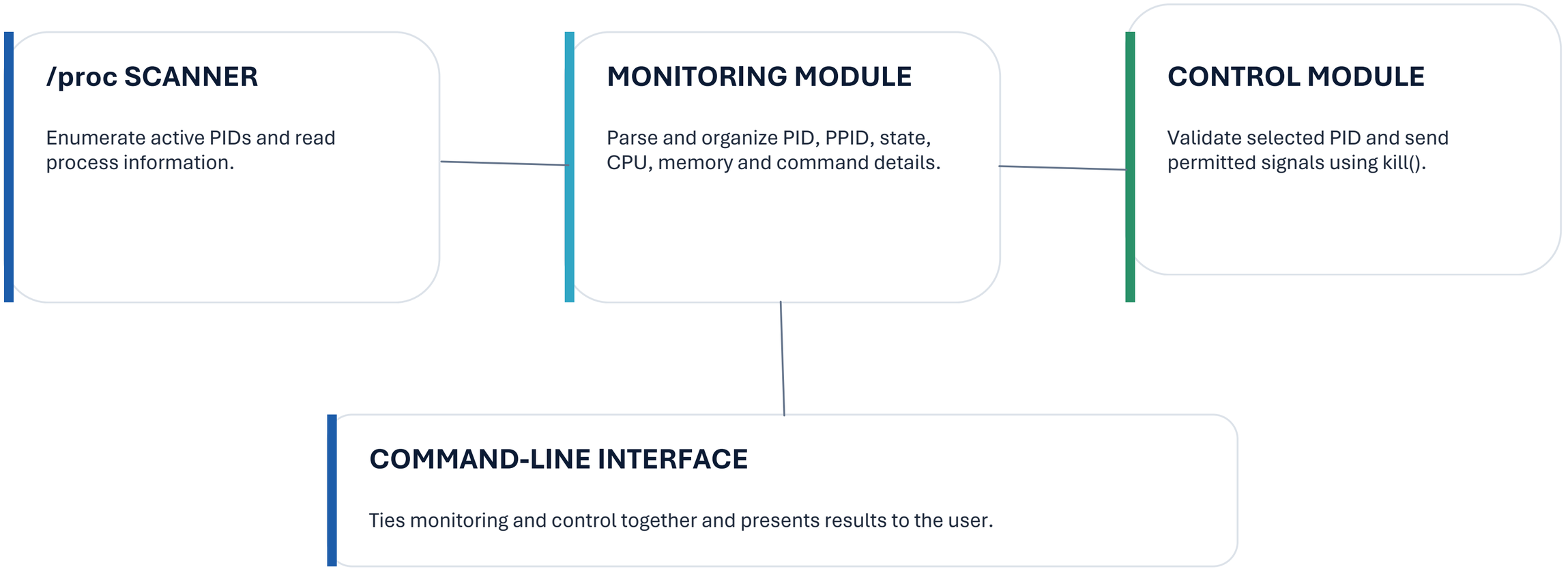
04

Demonstrate

Provide a clear Linux interface for safe, reliable monitoring and control.

System Architecture

Three core modules connect Linux process data, user commands and signal-based control.



Proposed Methodology

The implementation follows a repeatable scan → parse → display → control workflow.

1

SCAN

Scan /proc and enumerate numeric PID directories.

2

READ

Read /proc/[pid]/stat, /proc/[pid]/status and /proc/[pid]/cmdline.

3

PARSE

Extract and organize relevant process attributes.

4

DISPLAY

Present the process list in a clear table and refresh periodically.

5

CONTROL

Validate the selected PID and use kill() to send permitted signals.

What the Monitor Displays

The problem statement identifies the core attributes to collect from Linux.

PID	PPID	STATE	CPU	MEMORY	COMMAND
1248	1032	R	12.4%	2.1%	./process_monitor
1372	1248	S	1.7%	1.3%	bash
1410	1248	R	6.2%	0.8%	gcc main.c

PROCESS STATE

The project relates observed states to Linux process concepts.

RESOURCE USAGE

Basic CPU and memory usage are collected and displayed.

Controlled Process Management

Signals provide the control mechanism; validation and error handling are part of the design.

SELECT

User identifies a process using its PID.

VALIDATE

Check process existence and permission before sending a signal.

SIGNAL

Use kill() to send a permitted signal.

SIGTERM / SIGKILL

RESULT

Report success or an appropriate error.

Operating Systems Concepts & Linux APIs

PID / PPID

Identify processes and parent-child relationships.

Process States

Show whether a process is running, sleeping, stopped or terminated.

Signals / kill()

Send permitted control signals such as termination requests.

/proc filesystem

Read process and resource information exposed by Linux.

POSIX system calls

Interact with processes and obtain system-level information.

Process Scheduling

Relate observed CPU activity to OS scheduling concepts.

Command-Line Interface

A simple CLI connects monitoring, selection and process control.

```
process-monitor > list
PID     PPID    STATE   CPU   MEM   COMMAND
1248    1032    R       12%   2%    ./app
1372    1248    S       2%    1%    bash

process-monitor > signal 1248 SIGTERM
Signal sent successfully.
```

LIST

Present the active process list.

SELECT / CONTROL

Choose a PID and issue an allowed signal.

ERROR HANDLING

Surface invalid PID, missing process or permission errors.

Testing & Verification

The submitted methodology calls for sample workloads and comparison with standard Linux utilities.

METRIC CHECK

Compare reported metrics with standard utilities such as ps/top.

PROCESS LIFECYCLE

Create sample workloads and observe process appearance and termination.

EDGE CASES

Test invalid PIDs, exited processes and permission-related failures.

CONTROL SAFETY

Verify signal operations behave correctly under normal conditions.

Tools, Platforms & Software Used

- Linux / Ubuntu
- C / C++
- GCC / G++
- POSIX / Linux APIs
- /proc filesystem

Team Contributions

Responsibilities are taken directly from the submitted project form.

Adithya Vishnubatla	Process control and signal-handling module.
Abhinav Sunkara	Process monitoring module and /proc data collection.
Abhinav Sunkara	User interface, testing, and integration.
T. Harsha Vardhan	Documentation, GitHub, and demonstration.

Expected Outcome

FUNCTIONAL UTILITY

A working Linux process monitoring and control system.

PROCESS VISIBILITY

Organized information about active processes and resource usage.

CONTROL

Controlled signal-based process management with validation.

LEARNING OUTCOME

Practical demonstration of OS process management and system-level programming.

CORE IDEA

Conclusion

- Monitor live Linux processes using information exposed through /proc.
- Present PID, PPID, state, CPU, memory and command details clearly.
- Use Linux signals and kill() for controlled process-management operations.
- Demonstrate process, scheduling, signaling and resource-monitoring concepts in practice.

Thank you • Questions & Discussion